

NOXUS — WEEK 3 & WEEK 4

WEEK 3: DEVELOPMENT FOUNDATIONS & DOMAIN EXPLORATION

1. WEEK 3 OVERVIEW

Week Name: Development Foundations & Domain Exploration

Duration: 5 Days

Main Objective

After learning programming basics and problem-solving in Week 2, students now begin exploring how programming is actually used to build applications, systems, and products.

Students will be introduced to different technical domains and basic development tools before selecting a more focused learning direction.

2. WEEK 3 GOALS

By the end of Week 3, students should:

- Understand how software and technical products are built.
 - Explore different technical domains.
 - Select one or more preferred domains.
 - Learn the basics of APIs and libraries.
 - Understand frontend, backend, databases, and deployment.
 - Get exposure to electronics and hardware-related development paths.
 - Learn how to read documentation.
 - Learn how to use external libraries.
 - Build a small domain-based activity.
 - Continue coding practice and XP progression.
-

3. DAY-WISE PLAN

DAY 1 — HOW TECHNOLOGY PRODUCTS ARE BUILT

Training Topic

From Code to Real Product

Topics Covered

- How an idea becomes a product.
- Frontend.
- Backend.
- Database.
- APIs.
- Authentication.
- Deployment.
- Client-server architecture.
- Introduction to software development lifecycle.

Basic Flow

Idea



Design



Frontend



Backend



Database



Testing



Deployment

Activity

Students will select an existing application and identify:

- What could be the frontend?
- What could be the backend?
- What data needs to be stored?
- What APIs might be used?
- How the application could be deployed?

Outcome

Students understand the basic architecture behind modern applications.

DAY 2 — LIBRARIES, APIS & DOCUMENTATION

Training Topic

Learning How Developers Learn

Topics Covered

- What is a library?
- What is a framework?
- What is an API?
- How to install a library.
- How to read official documentation.
- Understanding dependencies.
- Package managers.
- Basic API requests and responses.

Example Concepts

```
Application
  ↓
API Request
  ↓
Server
  ↓
Data Processing
  ↓
API Response
  ↓
Application
```

Student Activity

Students will:

- Install one beginner-friendly library.
- Read basic documentation.
- Use the library in a small program.
- Make a simple API request where appropriate.

Tasks

- [] Complete library exercise.
- [] Complete API exercise.
- [] Document what was learned.

- [] Push the activity to GitHub.
-

DAY 3 — DOMAIN EXPLORATION

Training Topic

Explore Before You Specialize

Students will receive introductory sessions for different domains.

SOFTWARE / IT

- Web Development
- Mobile Development
- AI/ML
- Data Science
- Cybersecurity
- Cloud and DevOps

ELECTRONICS & TELECOMMUNICATION

- Embedded Systems
- IoT
- Robotics
- PCB Design
- VLSI
- Communication Systems

MECHANICAL / MECHATRONICS

- CAD
- Automation
- Robotics
- EV Technology
- Manufacturing
- Mechatronics

OTHER PATHS

- UI/UX
- Product Development
- Research
- Technical Content
- Entrepreneurship

Student Activity

Students select:

Primary Interest

Secondary Interest

Domains they want to explore further

Outcome

Noxus begins building an individual domain profile for every student.

DAY 4 — DOMAIN HANDS-ON LAB

Students complete a small hands-on activity based on their interests.

Web Development

Build a simple webpage.

AI/ML

Use a simple dataset or basic Python model.

IoT / Embedded

Create a basic sensor or microcontroller simulation.

Data Science

Analyze a small dataset.

UI/UX

Create a basic application interface or wireframe.

CAD / Mechanical

Create a simple model or design.

Robotics

Create a basic simulation or control logic.

Task

Students submit:

- Project/activity source.
- GitHub repository where applicable.
- Screenshots or output.

- Short README.
 - What they learned.
-

DAY 5 — DOMAIN CHALLENGE & REVIEW

Students participate in a small domain challenge.

Challenge Structure

Students are given a simple problem.

They must:

1. Understand the problem.
2. Select an appropriate domain.
3. Suggest a technical solution.
4. Identify required tools.
5. Create a basic prototype or implementation.
6. Present their approach.

Mentor Review

Mentors provide feedback on:

- Interest.
 - Participation.
 - Understanding.
 - Technical approach.
 - Documentation.
-

4. WEEK 3 CODING PRACTICE

Students continue their coding journey.

Practice Areas

- Arrays.
- Strings.
- Hashing basics.
- Searching.
- Basic recursion where appropriate.
- Problem-solving patterns.

Difficulty

The majority of problems should remain beginner-friendly.

Suggested progression:

- Easy practice problems.
- Topic-based problems.
- One weekly challenge.
- Optional advanced problems.

Students continue using their configured coding-platform-to-GitHub workflow where available.

5. WEEK 3 XP ACTIVITIES

Activity	Suggested XP
Complete training session	20 XP
Complete domain exploration	20 XP
Library/API activity	30 XP
Domain hands-on lab	50 XP
Coding problem	10–20 XP
Documentation	10 XP
Weekly challenge	50 XP
Complete Week 3	75 XP

6. WEEK 3 COMPLETION CRITERIA

Students must:

- ☐ Attend domain introduction sessions.
 - ☐ Complete the library exercise.
 - ☐ Complete the API exercise.
 - ☐ Explore available domains.
 - ☐ Select primary and secondary interests.
 - ☐ Complete one domain hands-on activity.
 - ☐ Submit documentation.
 - ☐ Continue coding practice.
 - ☐ Complete the weekly challenge.
-

7. WEEK 3 OUTCOME

By the end of Week 3, students should know:

- How modern applications are structured.
 - What APIs and libraries are.
 - How to read documentation.
 - How different technical domains work.
 - Which domains interest them.
 - How their programming skills can be applied to real projects.
-

WEEK 4: COLLABORATION, TEAM FORMATION & MINI PROJECT SPRINT

1. WEEK 4 OVERVIEW

Week Name: Collaboration, Team Formation & Mini Project Sprint

Duration: 5–7 Days

Main Objective

Week 4 moves students from **individual learning to collaborative building**.

Students will learn how technical teams work, how projects are planned, how responsibilities are divided, and how GitHub can be used for collaboration.

2. WEEK 4 GOALS

By the end of Week 4, students should:

- Understand technical teamwork.
 - Learn collaborative Git and GitHub workflows.
 - Form temporary or permanent mini-project teams.
 - Select a problem statement.
 - Plan a solution.
 - Divide responsibilities.
 - Build a working mini project.
 - Use issues, branches, and pull requests.
 - Document their contribution.
 - Present their project.
-

3. DAY 1 — TEAMWORK & COLLABORATION

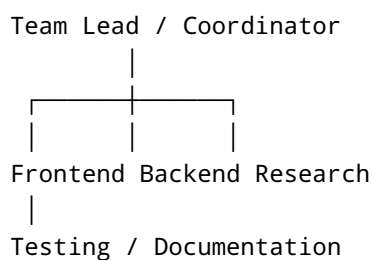
Training Topic

How Technical Teams Work

Topics Covered

- Team roles.
- Communication.
- Responsibility.
- Task ownership.
- Daily updates.
- Meetings.
- Working with mentors.
- Handling disagreements.
- Asking for help.
- Giving constructive feedback.

Example Team Structure



Students should understand that roles are flexible and members should learn from each other.

4. DAY 2 — COLLABORATIVE GITHUB

Training Topic

GitHub for Teams

Topics Covered

- Organization and repositories.
- Collaborators.
- Branches.
- Feature branches.
- Pull requests.
- Code review.
- Merge conflicts.
- Issues.

- Project boards.

Hands-On Activity

Students form small practice teams.

Each member:

1. Clones a shared repository.
2. Creates a branch.
3. Makes a small change.
4. Commits the change.
5. Pushes the branch.
6. Creates a pull request.
7. Reviews another member's code.
8. Merges the change.

Outcome

Students understand basic collaborative development.

5. DAY 3 — PROBLEM DISCOVERY & PROJECT PLANNING

Teams select a problem to solve.

Project Planning Template

Problem

What problem are we solving?

Target Users

Who experiences this problem?

Proposed Solution

How will the project solve it?

Required Technology

What tools, languages, hardware, or APIs are required?

Team Members

Who is responsible for what?

MVP

What is the minimum working version?

6. DAY 4 — MINI PROJECT DEVELOPMENT

Teams begin implementation.

Example Mini Projects

Web

- Student task manager.
- Event registration system.
- Expense tracker.
- Community dashboard.

AI/ML

- Simple recommendation system.
- Classification project.
- Basic prediction system.

IoT

- Temperature monitoring.
- Smart light control.
- Sensor dashboard.

Embedded Systems

- Basic automation system.
- Sensor-based control.
- Alert system.

UI/UX

- Mobile application prototype.
- Student platform redesign.

Mechanical / CAD

- Product prototype.

- Automation mechanism.
 - Basic component design.
-

7. DAY 5 — TESTING, PRESENTATION & REVIEW

Teams complete:

- Testing.
- Bug fixing.
- Documentation.
- README.
- Contribution summary.
- Basic presentation.

Final Presentation Structure

1. Problem.
 2. Why the problem matters.
 3. Proposed solution.
 4. Technology used.
 5. Team members.
 6. Live demonstration.
 7. Challenges faced.
 8. What the team learned.
 9. Future improvements.
-

8. WEEK 4 PROJECT REPOSITORY STRUCTURE

```
mini-project/  
|  
├── README.md  
├── docs/  
│   └── project-plan.md  
|  
├── src/  
|  
├── assets/  
|  
├── tests/  
|  
└── CONTRIBUTING.md
```

The README should contain:

- Project name.

- Problem statement.
 - Solution.
 - Features.
 - Technology stack.
 - Installation instructions.
 - Team members.
 - Screenshots/demo.
 - Future improvements.
-

9. TEAM CONTRIBUTION TRACKING

Noxus can track team participation through:

- Assigned tasks.
- Task completion.
- Relevant commits.
- Pull requests.
- Issues.
- Documentation.
- Peer feedback.
- Mentor review.

Again:

More commits should not automatically mean more contribution.

Contribution should consider:

Task Completion

-

Code/Work Quality

-

Documentation

-

Collaboration

-

Mentor Review

10. WEEK 4 XP SYSTEM

Activity	Suggested XP
Team formation	20 XP
GitHub collaboration lab	30 XP
Project planning	30 XP
Task completion	20–50 XP
Pull request accepted	20 XP
Documentation contribution	20 XP
Project presentation	50 XP
Complete mini project	150 XP
Complete Week 4	100 XP

Team Bonus

If every team member completes their assigned responsibilities, the entire team can receive a completion bonus.

This encourages collaboration instead of individual competition.

11. WEEK 4 STUDENT DASHBOARD

WEEK 4 – MINI PROJECT SPRINT

Team: Team Alpha

Project: Student Task Manager

Your Role: Backend

Tasks Assigned: 4

Tasks Completed: 3

Pull Requests: 2

Documentation: Completed

Team Progress: 75%

Personal Contribution: Active

12. WEEK 4 COMPLETION CHECKLIST

Every student must:

- ☐ Attend teamwork training.
 - ☐ Complete collaborative GitHub practice.
 - ☐ Join a mini-project team.
 - ☐ Participate in project planning.
 - ☐ Accept assigned responsibilities.
 - ☐ Complete assigned tasks.
 - ☐ Contribute to the project.
 - ☐ Participate in testing.
 - ☐ Update documentation.
 - ☐ Complete contribution summary.
 - ☐ Participate in the final presentation.
-

13. WEEK 4 FINAL OUTCOME

By the end of Week 4, students should have:

1. Experience working in a technical team.
 2. Experience using GitHub collaboratively.
 3. Experience with branches and pull requests.
 4. A completed mini project.
 5. A documented GitHub repository.
 6. A clear understanding of project development.
 7. Experience presenting a technical solution.
 8. Initial teamwork and contribution data stored in Noxus.
-

WEEK 3 & 4 JOURNEY

WEEK 3

Programming Knowledge



Libraries & APIs



Domain Exploration



Hands-On Domain Activity



Domain Challenge



Identify Interests

WEEK 4

Teamwork Training



Collaborative GitHub



Team Formation



Problem Selection



Project Planning



Development



Testing



Documentation



Presentation



READY FOR THE NEXT PHASE: DEEPER LEARNING, DOMAIN SPECIALIZATION & LARGER PROJECTS